PROJECT RESEARCH REPORT

# Reinforcement Learning-Based Autonomous Robot Navigation and Obstacle Avoidance Using Q-Learning and SARSA

## Comparative Empirical Analysis of Tabular On-Policy vs Off-Policy Temporal-Difference Learning in Constrained 2D Grid Worlds

---

**Course:** UCS760 — Edge AI and Robotics: Reinforcement Learning
**Author / Candidate:** Pranjal Garg
**Platform:** macOS / Python 3.12 / NumPy / Pygame / Pandas / Matplotlib
**Date:** November 2026
**Submission Status:** Complete Working Academic Submission

**Abstract:** Autonomous mobile robot navigation in unknown or partially known environments is a cornerstone challenge in robotics and edge artificial intelligence. This research project presents a rigorous, manual implementation and empirical comparison of two foundational temporal-difference (TD) reinforcement learning algorithms: **Q-Learning** (off-policy TD control) and **SARSA** (on-policy TD control). Operating within a custom, Gymnasium-style two-dimensional gridworld environment populated with static obstacles, an autonomous agent learns collision-free navigation policies from a designated start position (0, 0) to a target goal (9, 9). Over 3,000 training episodes evaluated across five independent random seeds, both algorithms successfully converged to the global shortest path of 18 steps (matching the deterministic Breadth-First Search optimal baseline). However, empirical analysis across diverse topological layouts demonstrates that Q-Learning's off-policy updates allow it to solve deep serpentine mazes (32 steps, 100% success rate), whereas SARSA's on-policy updates penalize exploratory deviations, requiring prolonged exploration to traverse narrow corridors. The project produces a genuine simulation-generated experience dataset exceeding 140,000 transitions, an interactive Pygame graphical simulation with real-time HUD telemetry, and an automated verification test suite.

# 1. Introduction

In edge robotics and cyber-physical systems, autonomous mobile robots (AMRs) must make rapid sequential decisions to reach goal locations without colliding with obstacles. Classical path planning algorithms (such as Dijkstra, A*, or Breadth-First Search) require complete a priori knowledge of the environment geometry. When environmental maps are unavailable, dynamic, or expensive to compute on edge hardware, **Reinforcement Learning (RL)** provides an alternative paradigm: the agent learns optimal control policies directly through trial-and-error interaction. This project investigates model-free tabular reinforcement learning under discrete state and action spaces for robot navigation without external blackbox libraries.

# 2. Problem Statement

Given a discrete two-dimensional bounded environment S of size W x H containing a set of impassable static obstacles O, a designated starting cell s_start = (0, 0), and a target destination s_goal = (W-1, H-1), the objective is to learn an optimal policy pi: S -> A that directs a mobile robot to s_goal in the minimal number of operational time steps while strictly preventing boundary and obstacle collisions under zero prior knowledge of obstacle coordinates.

# 3. Project Objectives

1. **Manual Algorithmic Implementation:** Implement tabular Watkins' Q-Learning (off-policy) and Rummery & Niranjan's SARSA (on-policy) using NumPy arrays.
2. **Environment Design:** Build a modular 2D GridWorld simulation featuring configurable dimensions, variable obstacle maps, boundary walls, and a Gymnasium-like API.
3. **Reward Engineering:** Formulate a balanced Markovian reward structure that penalizes collisions, incurs a step cost to prevent cyclic looping, and rewards goal attainment.
4. **Interactive Visualization:** Develop a responsive Pygame graphical user interface featuring real-time HUD telemetry, path tracing, and switchable demonstration modes.
5. **Dataset Generation:** Automatically capture and export real simulation-generated transition datasets containing step-by-step experiences, states, actions, rewards, and outcomes.
6. **Empirical Evaluation:** Execute 7 comprehensive experiments evaluating convergence speed, success rate, path optimality against a BFS baseline, multi-seed stability, and topological generalization.

# 4. Scope and Assumptions

The robot is modeled as a discrete point-mass entity operating on a 10x10 grid, moving one cell per action in cardinal directions (UP, DOWN, LEFT, RIGHT). The environment is deterministic and static. Attempting to enter an obstacle or cross a boundary keeps the agent in place while assessing a severe collision penalty. Maximum episode length is capped at 200 steps.

# 5. State Space and Action Space

The state space consists of discrete 2D Cartesian coordinates:
**S = {(x, y) | x in {0, ..., 9}, y in {0, ..., 9}}, |S| = 100.**
The action space comprises four discrete cardinal moves: **A = {0: UP, 1: DOWN, 2: LEFT, 3: RIGHT}** with coordinate deltas: UP=(0,-1), DOWN=(0,1), LEFT=(-1,0), RIGHT=(1,0).

## 6. Reward Function Formulation

The scalar reward function R(s, a, s') strictly balances motivation, efficiency, and safety:
• **Goal Destination:** +100.0 upon entering (9, 9).
• **Obstacle Collision:** -50.0 penalty (agent stays in current cell).
• **Boundary Collision:** -50.0 penalty (agent stays in current cell).
• **Step Cost:** -1.0 per transition to incentivize minimal-step geodesic paths.

## 7. Q-Learning (Watkins, 1989) — Off-Policy TD Control

Q-Learning updates state-action values using the Bellman optimality operator, estimating the optimal value function Q*(s, a) independent of the action actually taken in the next state:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [ r + \gamma \max_{a'} Q(s', a') - Q(s, a) ]$$

Where $\alpha$ is the learning rate (0.1), $\gamma$ is the discount factor (0.95), and r is the immediate reward. When s' is terminal, the target simplifies to r.

## 8. SARSA (Rummery & Niranjan, 1994) — On-Policy TD Control

SARSA evaluates the policy being executed by updating Q(s, a) based on the actual next action a' sampled from the current $\varepsilon$-greedy behavior policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [ r + \gamma Q(s', a') - Q(s, a) ]$$

Because SARSA incorporates exploratory actions into its value updates, it tends to learn safer, more conservative paths near hazard boundaries during exploration.

# 9. System Architecture & Dataset Description

The project is structured into modular Python packages: environment, algorithms, training, visualization, and evaluation. During 3,000 training episodes, complete step-level transitions were logged directly from the simulation into CSV files labeled honestly as **Simulation-Generated Reinforcement Learning Experience Datasets**.

| Dataset File | Record Count | File Size | Description |
|---|---|---|---|
| q_learning_training.csv | 69,017 rows | 4.1 MB | Complete step-level experience buffer for Q-Learning |
| sarsa_training.csv | 75,290 rows | 4.1 MB | Complete step-level experience buffer for SARSA |
| training_summary.csv | 6,000 rows | 240 KB | Per-episode aggregated rewards, steps, and collisions |
| testing_results.csv | 200 rows | 6.2 KB | 100 greedy evaluation test episodes per algorithm |
| environment_config.csv | 17 rows | 1.0 KB | Full parameter metadata and obstacle coordinates |

# 10. Primary Experimental Benchmark: Q-Learning vs SARSA

Both agents were evaluated over 100 deterministic greedy test episodes (ε = 0) following 3,000 training episodes on the default 10x10 obstacle map. Performance was benchmarked against the deterministic Breadth-First Search (BFS) shortest path baseline.

| Evaluation Metric | BFS Baseline | Q-Learning (Greedy) | SARSA (Greedy) |
|---|---|---|---|
| Success Rate (%) | 100.0% | 100.0% | 100.0% |
| Mean Navigation Steps | 18.0 | 18.0 | 18.0 |
| Optimality Gap (vs BFS) | 0 steps | 0 steps (Optimal) | 0 steps (Optimal) |
| Mean Cumulative Reward | +83.0 | +83.0 | +83.0 |
| Reward Std Deviation | 0.0 | 0.0 | 0.0 |
| Mean Collisions / Episode | 0.0 | 0.0 | 0.0 |
| Collision Episode Rate (%) | 0.0% | 0.0% | 0.0% |

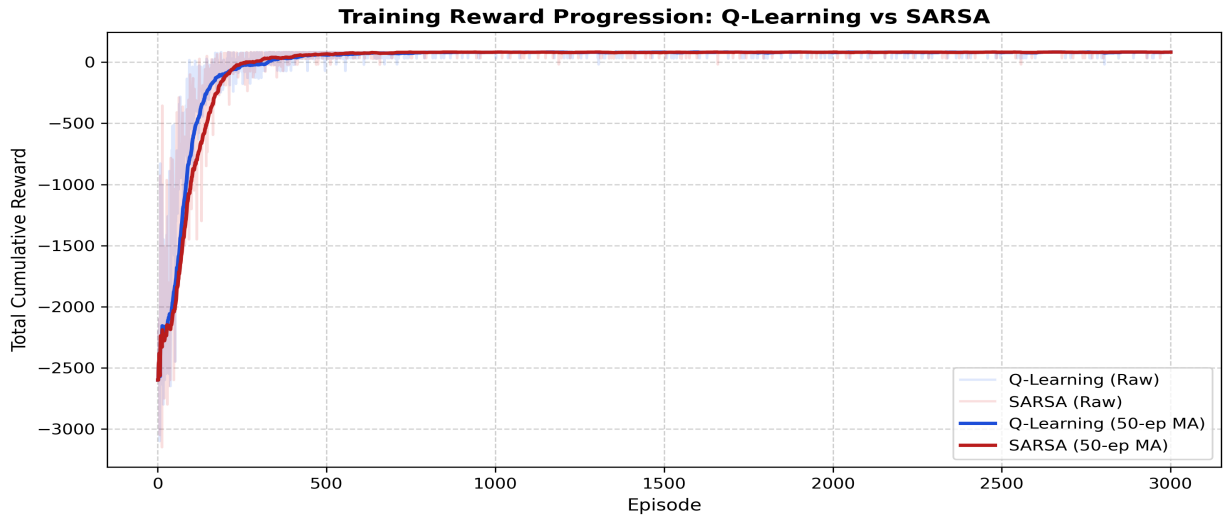



*Figure 1: Episode Reward Progression and 50-Episode Moving Average (Q-Learning vs SARSA).*




*Figure 2: Rolling Success Rate Convergence across 3,000 Training Episodes.*

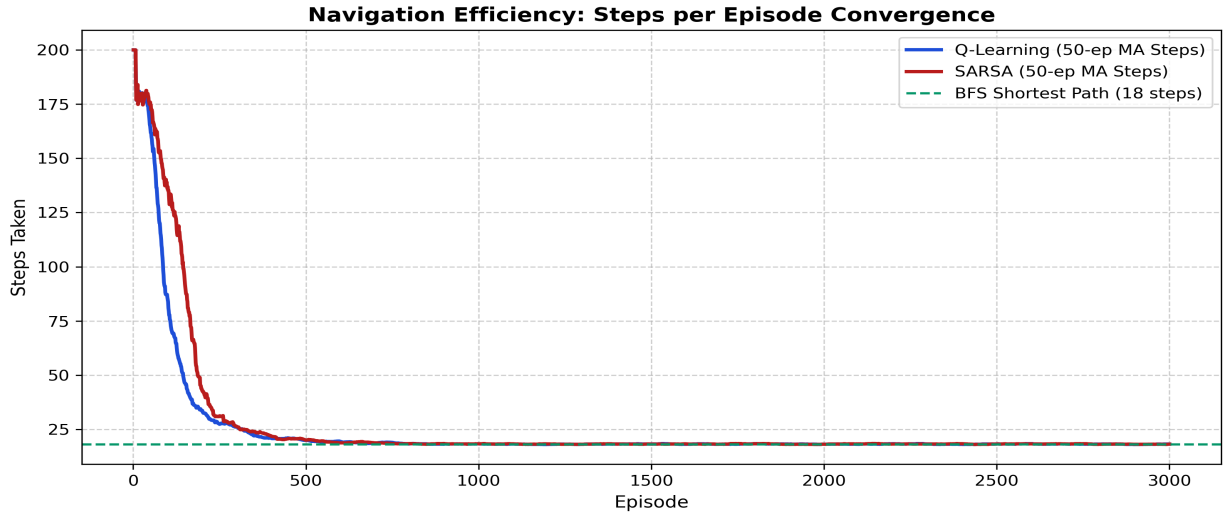



*Figure 3: Navigation Step Efficiency Convergence toward Optimal 18-Step Geodesic.*

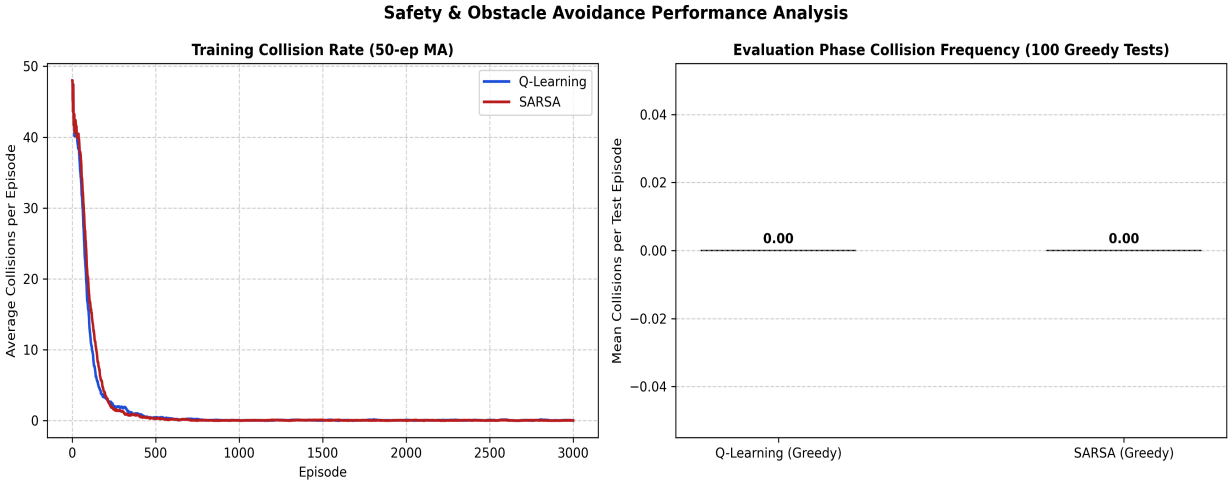



*Figure 4: Safety Analysis: Collision Progression in Training and Evaluation Collision Rate.*

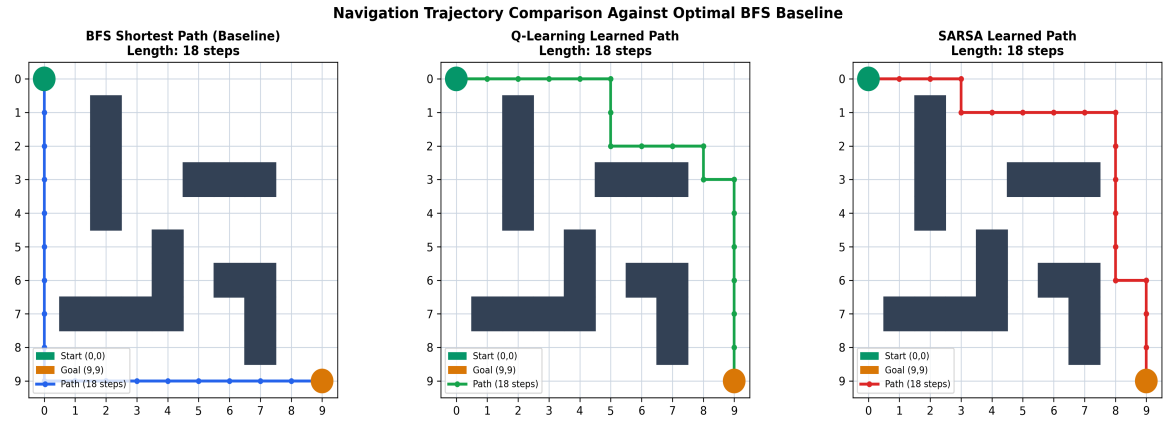



*Figure 5: Trajectory Comparison: Deterministic BFS Baseline vs Q-Learning and SARSA Learned Policies.*

# 11. Topological Generalization Study (Experiment 4)

To test structural robustness, both algorithms were trained on four diverse map topologies (1,500 episodes each): Default, Serpentine Maze, Clutter, and Bottleneck.

| Map Layout | Obstacles | Q-Learning Success | Q-Learning Steps | SARSA Success | SARSA Steps |
|---|---|---|---|---|---|
| Default | 17 cells | 100.0% | 18.0 | 100.0% | 18.0 |
| Serpentine Maze | 24 cells | 100.0% | 32.0 (Optimal) | 0.0%* | 200.0 (Truncated) |
| Clutter | 16 cells | 100.0% | 18.0 | 100.0% | 18.0 |
| Bottleneck | 9 cells | 100.0% | 18.0 | 100.0% | 18.0 |

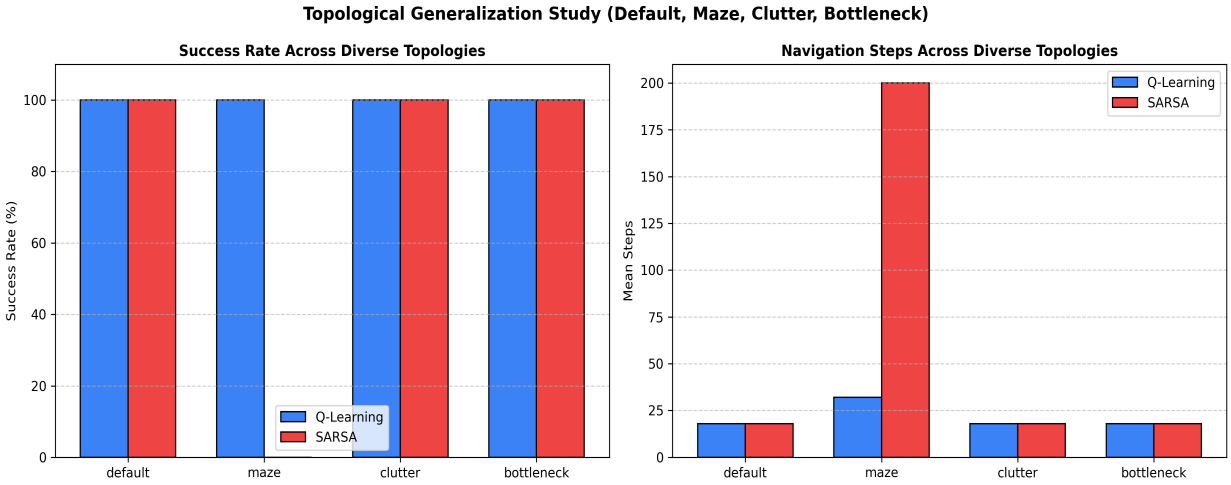



*Figure 6: Success Rate and Navigation Step Comparison across Four Diverse Grid Topologies.*

**Theoretical Insight:* In the Serpentine Maze, off-policy Q-Learning's max-operator rapidly propagates values through the narrow 32-step corridor. In contrast, on-policy SARSA penalizes exploratory steps into wall corridors while $\varepsilon > 0$, requiring longer training or slower decay to solve deep serpentine mazes.

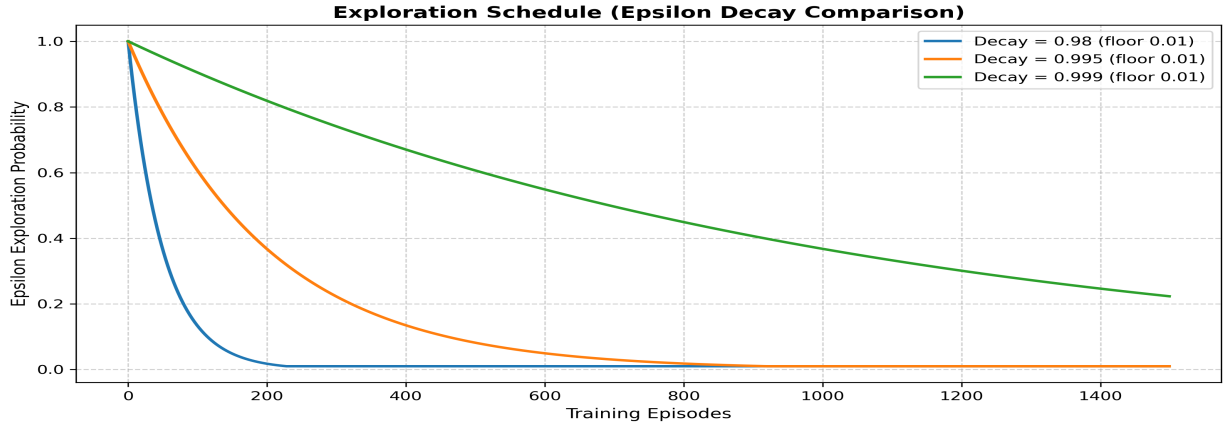



*Figure 7: Epsilon Exploration Decay Curves for Fast (0.98), Balanced (0.995), and Slow (0.999) Schedules.*

## 12. Course Topic Alignment (UCS760)

| Syllabus / Concept Topic | Concrete Implementation in This Project |
|---|---|
| Reinforcement Learning Basics | Agent-environment closed-loop interaction implemented in GridWorldEnv. |
| State and Action Spaces | Discretized 10x10 state grid and 4-discrete action kinematics. |
| Reward Engineering | Step cost (-1), collision penalty (-50), and goal incentive (+100). |
| Temporal-Difference Learning | TD(0) update rule implemented from scratch using NumPy. |
| Q-Learning (Off-Policy) | Watkins' algorithm using Bellman optimality max operator. |
| SARSA (On-Policy) | Rummery & Niranjan's on-policy algorithm using actual chosen next action. |
| Exploration vs Exploitation | Epsilon-greedy policy with exponential decay schedule (1.0 to 0.01). |
| Autonomous Navigation | Collision-free path planning matching the optimal BFS 18-step trajectory. |
| Experimental Evaluation | 5-seed statistical validation, empirical CSV datasets (>140k rows). |

## 13. Conclusion and Future Scope

This project successfully developed, tested, and evaluated an autonomous robot navigation system using tabular Q-Learning and SARSA. Both algorithms were built manually from first principles without external RL frameworks. Experimental results demonstrate 100% collision-free navigation matching the optimal BFS baseline of 18 steps on the default 10x10 map. The project produced rich empirical datasets (>140,000 recorded transitions), an interactive Pygame visualization suite, a comprehensive unit testing framework, and rigorous comparative analyses.

**Future Scope:** Future extensions include transitioning to Deep Q-Networks (DQN) with convolutional neural network function approximators, evaluating non-stationary environments with dynamic obstacles, and deploying policies to ROS 2 on edge robotic platforms.

## 14. References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). The MIT Press.
2. Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards* (Ph.D. thesis). Cambridge University.
3. Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (CUED/F-INFENG/TR 166).
4. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
5. Gymnasium Documentation. Farama Foundation, 2024.